

SQL MINI PROJECT

RETAIL STORE DATABASE

❖ LEVEL 01 → BASICS----->

61 # 1. Retrieve customer names and emails for email marketing--->

62

63 • SELECT NAME, EMAIL

64 FROM CUSTOMERS;

	NAME	EMAIL
▶	Thomas Owers	user1@example.com
	Charles Grant	user2@example.com
	Karlin Richards	user3@example.com
	Christna Williams	user4@example.com
	David Allen	user5@example.com
	Mark Duke	user6@example.com
	Briana Wright	user7@example.com
	John Bryan	user8@example.com
	Jason Thompson	user9@example.com
	Shawn Hill	user10@example.com
	Walter Jenkins	user11@example.com
	Mary Knight	user12@example.com

Toggle wrapping of cell contents

66 # 2. View complete product catalog with all available details--->

67

68 • SELECT *

69 FROM PRODUCTS;

	product_id	name	category	price	stock_quantity	added_on
▶	1	Plant No	Home	639.43	152	2024-01-30 06:30:53
	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	9	Common Study	Toys	985.19	171	2023-07-20 13:06:42
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	13	Full West	Books	2112.33	172	2023-09-15 03:13:38
	14	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
	15	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
	16	Good Thom	Toys	3073.06	110	2024-02-05 08:53:03

71 # 3. List all unique product categories--->

72

73 • SELECT DISTINCT CATEGORY

74 FROM PRODUCTS;

	CATEGORY
▶	Home
	Clothing
	Electronics
	Toys
	Books

76 # 4. Show all products priced above ₹1,000--->

77

78 • SELECT *

79 FROM PRODUCTS

80 WHERE PRICE > 1000;

	product_id	name	category	price	stock_quantity	added_on
▶	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	13	Full West	Books	2112.33	172	2023-09-15 03:13:38
	16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
	17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
	18	Good Thom	Toys	3073.06	110	2024-02-05 08:53:03

```

82 # 5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)--->
83
84 • SELECT *
85 FROM PRODUCTS
86 WHERE PRICE BETWEEN 2000 AND 5000;

```

	product_id	name	category	price	stock_quantity	added_on
▶	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	13	Full West	Books	2112.33	172	2023-09-15 03:13:38
	17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
	18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
	19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01

```

89 # 6. Fetch data for specific customer IDs (e.g., from loyalty program list)--->
90

```

```

91 • WITH CUSTOMER_ORDER_COUNTS AS (
92     SELECT
93         CUSTOMER_ID,
94         COUNT(ORDER_ID) AS TOTAL_ORDERS
95     FROM ORDERS
96     GROUP BY CUSTOMER_ID
97 ),
98 LOYALTY_CATEGORIZATION AS (
99     SELECT
100         CUSTOMER_ID,
101         TOTAL_ORDERS,
102         CASE
103             WHEN TOTAL_ORDERS >= 15 THEN 'LOYAL'
104             WHEN TOTAL_ORDERS >= 7 THEN 'POTENTIAL LOYAL'
105             WHEN TOTAL_ORDERS >= 3 THEN 'HIGH SPENDER'
106             ELSE 'RECENT DOINGERS'
107         END AS LOYALTY_PROGRAM
108     FROM CUSTOMER_ORDER_COUNTS
109 )
110 SELECT
111     CUSTOMER_ID,
112     TOTAL_ORDERS,
113     LOYALTY_PROGRAM
114 FROM LOYALTY_CATEGORIZATION;

```

	CUSTOMER_ID	TOTAL_ORDERS	LOYALTY_PROGRAM
▶	1	12	POTENTIAL LOYAL
	2	17	LOYAL
	3	17	LOYAL
	4	9	POTENTIAL LOYAL
	5	14	POTENTIAL LOYAL

```

115 # 7. Identify customers whose names start with the letter 'A'--->
116
117 • SELECT NAME , CUSTOMER_ID
118 FROM CUSTOMERS
119 WHERE NAME LIKE "A%";

```

	NAME	CUSTOMER_ID
▶	Austin Flores	15
	Amy Landry	16
	Amanda Bright	19
	Adrienne Green	27

```

21 # 8. List electronics products priced under ₹3,000--->
22
23 • SELECT *
24 FROM PRODUCTS
25 WHERE CATEGORY = "ELECTRONICS" AND PRICE < 3000;

```

	product_id	name	category	price	stock_quantity	added_on
▶	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
	15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
	31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
	34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45
	44	Actually Term	Electronics	396.11	85	2023-11-02 13:09:20
	47	Southern Thing	Electronics	512.46	40	2024-02-28 17:57:38

```

27 # 9. Display product names and prices in descending order of price--->
28
29 • SELECT NAME , PRICE
30 FROM PRODUCTS
31 ORDER BY PRICE DESC;

```

	NAME	PRICE
▶	Response Indeed	4897.36
	Population Social	4813.68
	Development System	4801.78
	Any Question	4759.28
	Fire Often	4734.89
	Natural Network	4722.66
	Build High	4707.14
	Serious Recognize	4523.10
	Study Total	4413.68

```

33 # 10. Display product names and prices, sorted by price and then by name--->
34
35 • SELECT NAME , PRICE
36 FROM PRODUCTS
37 ORDER BY PRICE ASC , NAME ASC;

```

	NAME	PRICE
▶	If Whatever	177.40
	Listen Development	296.17
	Actually Term	396.11
	Television Stock	421.73
	Southern Thing	512.46
	Plant No	639.43
	Place Low	723.97
	Stock Arbid	834.75
	Toward Minute	857.85

❖ LEVEL 02 → FILTERING & FORMATTING ----->

```

141 # 1. Retrieve orders where customer information is missing (possibly due to data migration or deletion)--->
142
143 • SELECT
144     O.ORDER_ID,
145     O.ORDER_DATE,
146     O.TOTAL_AMOUNT
147 FROM
148     ORDERS O
149 JOIN
150     CUSTOMERS C
151 ON
152     O.CUSTOMER_ID = C.CUSTOMER_ID
153 WHERE
154     O.CUSTOMER_ID IS NULL
155     OR TRIM(O.CUSTOMER_ID) = ''
156     OR C.CUSTOMER_ID IS NULL;

```

ORDER_ID	ORDER_DATE	TOTAL_AMOUNT
----------	------------	--------------

```

158 # 2. Display customer names and emails using column aliases for frontend readability--->
159
160 • SELECT
161     NAME AS CUSTOMER_NAME,
162     EMAIL AS CUSTOMER_EMAIL
163 FROM
164     CUSTOMERS;

```

	CUSTOMER_NAME	CUSTOMER_EMAIL
▶	Thomas Owens	user1@example.com
	Charles Grant	user2@example.com
	Kaitlin Richards	user3@example.com
	Christina Williams	user4@example.com
	David Allen	user5@example.com
	Mark Duke	user6@example.com
	Briana Wright	user7@example.com
	John Bryan	user8@example.com
	Jason Thompson	user9@example.com
	Shawn Hill	user10@example.com
	Walter Jenkins	user11@example.com

```

166 # 3. Calculate total value per item ordered by multiplying quantity and item price--->
167
168 • SELECT
169     ORDER_ITEM_ID,
170     SUM(QUANTITY * ITEM_PRICE) AS TOTAL_VALUE
171 FROM
172     ORDER_ITEMS
173 GROUP BY
174     ORDER_ITEM_ID;

```

	ORDER_ITEM_ID	TOTAL_VALUE
▶	1	9414.28
	2	532.20
	3	2955.57
	4	2208.99
	5	9469.78
	6	4707.14
	7	9794.72
	8	14692.08
	9	1429.45
	10	723.97
	11	14204.67

```

176 # 4. Combine customer name and phone number in a single column--->
177
178 • SELECT
179     CONCAT(NAME , "---->" , PHONE) AS CUSTOMER_DETAILS
180 FROM
181     CUSTOMERS;

```

	CUSTOMER_DETAILS
▶	Thomas Owens---->142-479-1945
	Charles Grant---->9153947511
	Kaitlin Richards---->2073473421
	Christina Williams---->505-605-5061x06
	David Allen---->(751)456-8289x1
	Mark Duke---->(144)957-2811
	Briana Wright---->223-833-9635
	John Bryan---->045.568.0798x27
	Jason Thompson---->1862659420
	Shawn Hill---->(268)113-3152x7

```

183 # 5. Extract only the date part from order timestamps for date-wise reporting--->
184
185 • SELECT
186     DATE(ORDER_DATE) AS CUSTOMER_ORDER_DATE,
187     COUNT(*) AS TOTAL_ORDER
188 FROM
189     ORDERS
190 GROUP BY
191     DATE(ORDER_DATE)
192 ORDER BY
193     CUSTOMER_ORDER_DATE DESC;

```

	CUSTOMER_ORDER_DATE	TOTAL_ORDER
▶	2025-06-13	2
	2025-06-11	3
	2025-06-10	2
	2025-06-09	3
	2025-06-08	1
	2025-06-06	5
	2025-06-05	1

```

195 # 6. List products that do not have any stock left--->
196
197 • SELECT
198     PRODUCT_ID,
199     NAME,
200     STOCK_QUANTITY
201 FROM
202     PRODUCTS
203 WHERE
204     STOCK_QUANTITY = 0;

```

PRODUCT_ID	NAME	STOCK_QUANTITY
------------	------	----------------

❖ LEVEL 03 → AGGREGATION ----->

```

206 # LEVEL 03---AGGREGATION----->
207 # 1. Count the total number of orders placed--->
208
209 • SELECT COUNT(*) AS TOTAL_ORDER
210 FROM ORDERS;

```

TOTAL_ORDER

▶ 400

```

212 # 2. Calculate the total revenue collected from all orders--->
213
214 • SELECT SUM(AMOUNT_PAID) AS TOTAL_REVENUE
215 FROM PAYMENTS;

```

TOTAL_REVENUE

▶ 6960973.66

```

217 # 3. Calculate the average order value--->
218
219 • SELECT AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE
220 FROM ORDERS;

```

AVERAGE_ORDER_VALUE

▶ 17402.434150

```

222 # 4. Count the number of customers who have placed at least one order--->
223
224 • SELECT
225     COUNT(DISTINCT CUSTOMER_ID) AS NUMBER_OF_CUSTOMER
226 FROM
227     ORDERS;

```

NUMBER_OF_CUSTOMER

▶ 30

233 # 5. Find the number of orders placed by each customer--->

```
234
235 • SELECT
236     DISTINCT COUNT(ORDER_ID) AS NUMBER_OF_ORDERS , CUSTOMER_ID
237 FROM
238     ORDERS
239 GROUP BY
240     CUSTOMER_ID;
```

	NUMBER_OF_ORDERS	CUSTOMER_ID
▶	12	1
	17	2
	17	3
	9	4
	14	5
	16	6
	13	7
	10	8
	10	9
	13	10
	9	11
	11	12

242 # 6. Find total sales amount made by each customer--->

```
243
244 • SELECT
245     DISTINCT SUM(TOTAL_AMOUNT) AS SALES_AMOUNT_PER_CUSTOMER , CUSTOMER_ID
246 FROM
247     ORDERS
248 GROUP BY
249     CUSTOMER_ID;
```

	SALES_AMOUNT_PER_CUSTOMER	CUSTOMER_ID
▶	183747.44	1
	284420.07	2
	253783.31	3
	137562.22	4
	262504.19	5
	212173.58	6
	167960.11	7
	164701.78	8
	106226.03	9
	252722.75	10
	173409.01	11

250 # 7. List the number of products sold per category--->

```
251
252
253 • SELECT
254     DISTINCT COUNT(PRODUCT_ID) , CATEGORY
255 FROM
256     PRODUCTS
257 GROUP BY
258     CATEGORY;
```

	COUNT(PRODUCT_ID)	CATEGORY
▶	8	Home
	13	Clothing
	14	Electronics
	8	Toys
	7	Books

260 # 8. Find the average item price per category--->

```
261
262 • SELECT
263     ROUND(AVG(PRICE),2) , CATEGORY
264 FROM
265     PRODUCTS
266 GROUP BY
267     CATEGORY;
```

	ROUND(AVG(PRICE),2)	CATEGORY
▶	2146.37	Home
	3434.58	Clothing
	2653.39	Electronics
	2516.53	Toys
	3167.08	Books

```

269 # 9. Show number of orders placed per day--->
270
271 • SELECT
272     COUNT(DISTINCT ORDER_ID) AS NUMBER_OF_ORDER , ORDER_DATE
273 FROM
274     ORDERS
275 GROUP BY
276     ORDER_DATE
277 ORDER BY
278     ORDER_DATE DESC;

```

	NUMBER_OF_ORDER	ORDER_DATE
▶	1	2025-06-13 23:21:01
	1	2025-06-13 10:58:21
	1	2025-06-11 13:30:19
	1	2025-06-11 08:36:00
	1	2025-06-11 03:03:58
	1	2025-06-10 18:01:32
	1	2025-06-10 17:00:25
	1	2025-06-09 19:00:34
	1	2025-06-09 13:59:27
	1	2025-06-09 06:26:55

```

280 # 10. List total payments received per payment method--->
281
282 • SELECT
283     SUM(DISTINCT AMOUNT_PAID) AS TOTAL_PAYMENTS , METHOD
284 FROM
285     PAYMENTS
286 GROUP BY
287     METHOD;

```

	TOTAL_PAYMENTS	METHOD
▶	1745188.82	Credit Card
	1928908.38	Debit Card
	1657367.22	Net Banking
	1594211.48	UPI

❖ LEVEL 04 → TABLE QUERIES(JOINS) ----->

```

285 # LEVEL 04---MULTI-TABLE QUERRIES (JOINS)----->
286 # 1. Retrieve order details along with the customer name (INNER JOIN)--->
287
288 • SELECT
289     O.ORDER_ID,
290     O.ORDER_DATE,
291     O.TOTAL_AMOUNT,
292     O.STATUS,
293     C.NAME
294 FROM
295     ORDERS O
296 JOIN
297     CUSTOMERS C
298 ON
299     O.CUSTOMER_ID = C.CUSTOMER_ID;

```

	ORDER_ID	ORDER_DATE	TOTAL_AMOUNT	STATUS	NAME
▶	14	2024-09-24 21:21:38	15803.34	Shipped	Thomas Owens
	17	2024-08-19 21:17:57	11173.77	Pending	Thomas Owens
	61	2024-12-26 23:21:58	13053.00	Shipped	Thomas Owens
	76	2025-01-14 22:59:54	23506.81	Pending	Thomas Owens
	92	2024-09-26 11:33:58	834.75	Shipped	Thomas Owens
	109	2025-03-17 04:56:18	12190.14	Shipped	Thomas Owens
	127	2025-06-10 18:01:32	20160.64	Pending	Thomas Owens
	135	2025-03-11 22:58:23	8891.79	Pending	Thomas Owens
	144	2024-09-19 09:18:22	12093.54	Shipped	Thomas Owens
	221	2024-09-25 05:49:39	12437.61	Pending	Thomas Owens
	278	2025-02-01 20:05:38	32015.16	Delivered	Thomas Owens

```
301 # 2. Get list of products that have been sold (INNER JOIN with order_items)--->
302
303 • SELECT
304     DISTINCT P.PRODUCT_ID,
305     P.NAME
306 FROM
307     PRODUCTS P
308 JOIN
309     ORDER_ITEMS OI
310 ON
311     P.PRODUCT_ID = OI.PRODUCT_ID;
```

	PRODUCT_ID	NAME
	45	Suffer We
	46	Age Treatment
	47	Southern Thing
	48	Group But
	49	Rest Something
	50	Stock Article

```
313 # 3. List all orders with their payment method (INNER JOIN)--->
314
315 • SELECT
316     DISTINCT O.ORDER_ID,
317     P.METHOD
318 FROM
319     ORDERS O
320 JOIN
321     PAYMENTS P
322 ON
323     O.ORDER_ID = P.ORDER_ID;
```

	ORDER_ID	METHOD
▶	1	Credit Card
	2	Net Banking
	3	Credit Card
	4	UPI
	5	UPI
	6	UPI
	7	UPI
	8	Net Banking

```
325 # 4. Get list of customers and their orders (LEFT JOIN)--->
326
327 • SELECT
328     C.CUSTOMER_ID,
329     C.NAME,
330     O.ORDER_ID,
331     O.ORDER_DATE
332 FROM
333     CUSTOMERS C
334 LEFT JOIN
335     ORDERS O
336 ON
337     C.CUSTOMER_ID = O.CUSTOMER_ID;
```

	CUSTOMER_ID	NAME	ORDER_ID	ORDER_DATE
▶	1	Thomas Owens	14	2024-09-24 21:21:38
	1	Thomas Owens	17	2024-08-19 21:17:57
	1	Thomas Owens	61	2024-12-26 23:21:58
	1	Thomas Owens	76	2025-01-14 22:59:54
	1	Thomas Owens	92	2024-09-26 11:33:58

```
339 # 5. List all products along with order item quantity (LEFT JOIN)--->
340
341 • SELECT
342     DISTINCT P.PRODUCT_ID,
343     P.NAME,
344     OII.QUANTITY
345 FROM
346     PRODUCTS P
347 LEFT JOIN
348     ORDER_ITEMS OII
349 ON
350     P.PRODUCT_ID = OII.PRODUCT_ID;
```

	PRODUCT_ID	NAME	QUANTITY
▶	1	Plant No	1
	1	Plant No	3
	1	Plant No	2
	2	Population Social	2
	2	Population Social	3
	2	Population Social	1
	3	Available Answer	1

```
352 # 6. List all payments including those with no matching orders (RIGHT JOIN)--->
353
354 • SELECT
355     P.PAYMENT_ID,
356     P.PAYMENT_DATE,
357     P.AMOUNT_PAID,
358     O.ORDER_ID,
359     O.ORDER_DATE
360 FROM
361     ORDERS O
362 RIGHT JOIN
363     PAYMENTS P
364 ON
365     O.ORDER_ID = P.ORDER_ID;
```

	PAYMENT_ID	PAYMENT_DATE	AMOUNT_PAID	ORDER_ID	ORDER_DATE
▶	1	2025-03-02 08:16:11	9414.28	1	2025-03-02 07:20:11
	2	2024-10-09 19:00:21	532.20	2	2024-10-09 18:08:21
	3	2025-05-08 00:55:27	5164.56	3	2025-05-08 00:08:27
	4	2024-09-19 22:28:13	9469.78	4	2024-09-19 22:16:13
	5	2025-04-08 18:26:06	14501.86	5	2025-04-08 18:02:06

```
367 # 7. Combine data from three tables: customer, order, and payment--->
368
369 • SELECT
370     C.CUSTOMER_ID,
371     C.NAME,
372     O.ORDER_ID,
373     O.ORDER_DATE,
374     P.PAYMENT_ID,
375     P.PAYMENT_DATE,
376     P.AMOUNT_PAID
377 FROM
378     CUSTOMERS C
379 JOIN
380     ORDERS O
381 ON
382     C.CUSTOMER_ID = O.CUSTOMER_ID
383 JOIN
384     PAYMENTS P
385 ON
386     O.ORDER_ID = P.ORDER_ID;
```

	CUSTOMER_ID	NAME	ORDER_ID	ORDER_DATE	PAYMENT_ID	PAYMENT_DATE	AMOUNT_PAID
▶	1	Thomas Owens	14	2024-09-24 21:21:38	14	2024-09-24 22:11:38	15803.34
	1	Thomas Owens	17	2024-08-19 21:17:57	17	2024-08-19 21:49:57	11173.77
	1	Thomas Owens	61	2024-12-26 23:21:58	61	2024-12-26 23:31:58	13053.00
	1	Thomas Owens	76	2025-01-14 22:59:54	76	2025-01-14 23:33:54	23506.81
	1	Thomas Owens	92	2024-09-26 11:33:58	92	2024-09-26 12:03:58	834.75
	1	Thomas Owens	109	2025-03-17 04:56:18	109	2025-03-17 05:16:18	12190.14
	1	Thomas Owens	127	2025-06-10 18:01:32	127	2025-06-10 18:57:32	20160.64

❖ LEVEL 05 → SUBQUERIES (INNER QUERIES) ----->

```
388 # LEVEL 05---SUBQUERIES(INNER QUERIES)----->
389 # 1. List all products priced above the average product price--->
390
391 * SELECT
392     PRODUCT_ID , NAME , PRICE
393 FROM
394     PRODUCTS
395 WHERE
396     PRICE > (
397         SELECT AVG(PRICE)
398         FROM PRODUCTS
399     )
400 ORDER BY PRICE DESC;
```

	PRODUCT_ID	NAME	PRICE
▶	7	Response Indeed	4897.36
	2	Population Social	4813.68
	10	Development System	4801.78
	4	Any Question	4759.28
	35	Fire Often	4734.89

```
402 # 2. Find customers who have placed at least one order--->
403
404 * SELECT
405     CUSTOMER_ID , NAME
406 FROM
407     CUSTOMERS
408 WHERE
409     CUSTOMER_ID
410 IN (
411     SELECT CUSTOMER_ID
412     FROM ORDERS
413 );
```

	customer_id	name
▶	1	Thomas Owens
	2	Charles Grant
	3	Kaitlin Richards
	4	Christina Williams
	5	David Allen
	6	Mark Duke

```
415 # 3. Show orders whose total amount is above the average for that customer--->
416
417 * SELECT
418     O1.ORDER_ID , O1.CUSTOMER_ID , O1.TOTAL_AMOUNT
419 FROM
420     ORDERS O1
421 WHERE
422     O1.TOTAL_AMOUNT > (
423         SELECT AVG(O2.TOTAL_AMOUNT)
424         FROM ORDERS O2
425         WHERE O2.CUSTOMER_ID = O1.CUSTOMER_ID
426     );
```

	ORDER_ID	CUSTOMER_ID	TOTAL_AMOUNT
▶	6	29	31050.17
	8	19	32714.06
	9	6	24219.20
	10	28	24342.52
	14	1	15803.34
	16	5	22856.73
	18	13	32001.24

```
428 # 4. Display customers who haven't placed any orders--->
429
430 * SELECT
431     CUSTOMER_ID , NAME
432 FROM
433     CUSTOMERS
434 WHERE
435     CUSTOMER_ID
436 NOT IN (
437     SELECT CUSTOMER_ID
438     FROM ORDERS
439 );
```

	CUSTOMER_ID	NAME
--	-------------	------

```

441 # 5. Show products that were never ordered--->
442
443 • SELECT
444     PRODUCT_ID , NAME
445 FROM
446     PRODUCTS
447 WHERE
448     PRODUCT_ID
449     NOT IN (
450         SELECT PRODUCT_ID
451         FROM ORDER_ITEMS
452         WHERE PRODUCT_ID IS NOT NULL
453     );

```

PRODUCT_ID	NAME
------------	------

```

455 # 6. Show highest value order per customer--->
456
457 • SELECT
458     O.CUSTOMER_ID , O.ORDER_ID , O.TOTAL_AMOUNT
459 FROM
460     ORDERS O
461 JOIN (
462     SELECT CUSTOMER_ID , MAX(TOTAL_AMOUNT) AS MAX_ORDER_VALUE
463     FROM ORDERS
464     GROUP BY CUSTOMER_ID
465 ) MO
466 ON O.CUSTOMER_ID = MO.CUSTOMER_ID
467 AND O.TOTAL_AMOUNT = MO.MAX_ORDER_VALUE;

```

	CUSTOMER_ID	ORDER_ID	TOTAL_AMOUNT
▶	13	18	32001.24
	8	33	36147.09
	10	62	35723.17
	27	66	26176.05
	24	68	36159.92

```

469 # 7. Highest Order Per Customer (Including Names)--->
470
471 • SELECT
472     C.NAME,
473     O.ORDER_ID,
474     O.TOTAL_AMOUNT
475 FROM
476     ORDERS O
477 JOIN (
478     SELECT CUSTOMER_ID , MAX(TOTAL_AMOUNT) AS MAX_ORDERVALUE
479     FROM ORDERS
480     GROUP BY CUSTOMER_ID
481 ) MO
482 ON O.CUSTOMER_ID = MO.CUSTOMER_ID
483 AND O.TOTAL_AMOUNT = MO.MAX_ORDERVALUE
484 JOIN CUSTOMERS C
485 ON O.CUSTOMER_ID = C.CUSTOMER_ID;

```

	NAME	ORDER_ID	TOTAL_AMOUNT
▶	Thomas Owens	278	32015.16
	Charles Grant	315	42056.04
	Kaitlin Richards	281	41679.11
	Christina Williams	386	25747.34
	David Allen	266	39921.78
	Mark Duke	330	39003.19
	Briana Wright	128	28589.04

❖ LEVEL 06 → SET OPERATIONS ----->

```
488 # 1. List all customers who have either placed an order or written a product review--->
489
490 • SELECT
491     CUSTOMER_ID , NAME , EMAIL
492 FROM
493     CUSTOMERS
494 WHERE CUSTOMER_ID IN (
495     SELECT CUSTOMER_ID
496     FROM ORDERS
497     UNION
498     SELECT CUSTOMER_ID
499     FROM PRODUCT_REVIEWS
500 );
```

	CUSTOMER_ID	NAME	EMAIL
▶	1	Thomas Owens	user1@example.com
	2	Charles Grant	user2@example.com
	3	Kaitlin Richards	user3@example.com
	4	Christina Williams	user4@example.com
	5	David Allen	user5@example.com

```
502 # 2. List all customers who have placed an order as well as reviewed a product [intersect not supported]--->
503
504 • SELECT
505     C.CUSTOMER_ID,
506     C.NAME,
507     C.EMAIL
508 FROM CUSTOMERS C
509 WHERE CUSTOMER_ID IN (
510     SELECT CUSTOMER_ID
511     FROM (
512         SELECT DISTINCT CUSTOMER_ID
513         FROM ORDERS
514         UNION ALL
515         SELECT DISTINCT CUSTOMER_ID
516         FROM PRODUCT_REVIEWS
517     ) AS COMBINED
518     GROUP BY CUSTOMER_ID
519     HAVING COUNT(*) = 2
520 );
```

	CUSTOMER_ID	NAME	EMAIL
▶	1	Thomas Owens	user1@example.com
	2	Charles Grant	user2@example.com
	4	Christina Williams	user4@example.com
	6	Mark Duke	user6@example.com
	7	Briana Wright	user7@example.com
	9	Jason Thompson	user9@example.com